

Atividade 1:

Funções no Mundo Real (Conceito e Analogia)

Objetivo: Entender o conceito de uma função através de exemplos do cotidiano, antes mesmo de escrever código.

Guia Detalhado: Uma função é, essencialmente, um **processo organizado** que recebe **entradas (inputs)**, realiza uma série de passos, e produz uma **saída (output)**. Nosso mundo está cheio de funções.

Seu Desafio: Para cada um dos exemplos abaixo, descreva as Entradas, o Processo e a Saída.

1. Exemplo: Fazer um sanduíche.

- **Entradas:** Pão, queijo, presunto, alface, etc.
- **Processo:** Pegar uma fatia de pão, colocar o queijo, depois o presunto, a alface e, por fim, a outra fatia de pão.
- **Saída:** Um sanduíche pronto.

2. Sua Vez: Sacar dinheiro em um caixa eletrônico.

- **Entradas:** _____ (Dica: O que você insere ou digita?)
- **Processo:** _____ (Dica: O que a máquina faz internamente?)
- **Saída:** _____ (Dica: O que você recebe no final?)

3. Sua Vez: Enviar uma mensagem de áudio no WhatsApp.

- **Entradas:** _____
- **Processo:** _____
- **Saída:** _____

Conclusão da Atividade: Você acabou de pensar como um programador! Nós criamos funções para encapsular processos e torná-los reutilizáveis.

Atividade 2:

Sua Primeira Função (Declaração e Chamada)

Objetivo: Aprender a sintaxe básica para criar (declarar) e usar (chamar) uma função que realiza uma ação simples.

Guia Detalhado: Vamos criar nossa primeira função de verdade. Ela não terá entradas nem saídas, será apenas um bloco de código nomeado que podemos executar. Esse tipo de função, que apenas *faz* algo, é chamado de **procedimento**.

Seu Desafio: Criar uma função que exibe uma linha de um poema na tela.

Código Guiado:

// 1. DECLARAÇÃO DA FUNÇÃO

// Usamos a palavra "funcao", damos um nome (verbo), e abrimos e fechamos parênteses e chaves.

```
funcao exibirVerso() {  
    escreva("Minha terra tem palmeiras,")  
    escreva("Onde canta o Sabiá;")  
}
```

// 2. O CÓDIGO AQUI FORA É O FLUXO PRINCIPAL DO PROGRAMA

```
escreva("Iniciando o programa...")
```

// 3. CHAMADA DA FUNÇÃO

// Para executar o código dentro da função, nós a chamamos pelo nome.

```
exibirVerso()
```

```
escreva("Programa finalizado.")
```

Para Refletir: O que aconteceria se você nunca chamasse a função `exibirVerso()`? O verso apareceria na tela? Por quê?

Atividade 3: Dando "Ingredientes" à Função (Parâmetros)

Objetivo: Tornar nossas funções mais úteis e flexíveis, permitindo que elas recebam dados (argumentos) através de parâmetros.

Guia Detalhado: Uma função que faz sempre a mesma coisa é pouco útil. Vamos modificar nossa saudação para que ela possa saudar qualquer pessoa. Para isso, criamos "espaços reservados" na declaração da função, chamados **parâmetros**. Quando chamamos a função, preenchemos esses espaços com valores reais, chamados **argumentos**.

- **Parâmetro:** A variável na declaração (nome).
- **Argumento:** O valor que você passa na chamada ("Djalma").

Código Guiado:

// Na declaração, "nomeUsuario" é um PARÂMETRO. É uma gaveta vazia.

```
funcao saudacaoPersonalizada( nomeUsuario ) {  
    escreva("Olá, " + nomeUsuario + "! Seja bem-vindo(a) ao sistema.")  
}
```

// Na chamada, "Ana" é um ARGUMENTO. É o que colocamos na gaveta.

```
saudacaoPersonalizada("Ana")
```

// Podemos reutilizar a função com um argumento diferente.

```
saudacaoPersonalizada("Bruno")
```

Seu Desafio: Crie uma função chamada `mostrarDobro` que recebe um número como parâmetro e exibe na tela a mensagem "O dobro de [número] é [resultado]". Chame-a com os números 5, 10 e 25.

Atividade 4:

Recebendo a "Encomenda" (O Comando retornar)

Objetivo: Aprender a criar funções que calculam algo e **devolvem** o resultado para que possamos usá-lo em outra parte do programa.

Guia Detalhado: Até agora, nossas funções apenas exibiram coisas na tela. Mas e se precisarmos do resultado de um cálculo para continuar nosso programa? Para isso, usamos a palavra-chave retornar. Ela "entrega" um valor de volta para quem chamou a função.

Código Guiado:

```
// Esta função NÃO exibe nada. Ela apenas calcula e RETORNA o resultado.
```

```
funcao somar(numeroA, numeroB) {  
    resultadoDaSoma = numeroA + numeroB  
    retornar resultadoDaSoma  
}
```

```
// Agora, no nosso programa principal, chamamos a função e  
GUARDAMOS o valor retornado.
```

```
valorGuardado = somar(10, 5)
```

```
escreva("Chamei a função somar com 10 e 5.")
```

```
escreva("O valor que ela me retornou foi: " + valorGuardado)
```

```
// Podemos usar o retorno diretamente em outras operações!
```

```
novaSoma = somar(valorGuardado, 3) // Estamos somando 15 + 3
```

```
escreva("O resultado de uma nova soma é: " + novaSoma)
```

Seu Desafio:

Crie uma função `calcularAreaRetangulo(largura, altura)` que receba duas medidas, calcule a área ($\text{largura} * \text{altura}$) e retorne o resultado. Depois, chame a função, guarde o resultado em uma variável e exiba a mensagem: "A área de um retângulo [largura]x[altura] é [resultado]".

Atividade 5:

Lidando com Múltiplos Tipos de Dados

Objetivo: Criar uma função mais complexa que lida com diferentes tipos de dados (texto, número, booleano) e usa lógica condicional.

Guia Detalhado: Funções podem receber qualquer tipo de dado. Vamos criar uma que gera um relatório sobre um produto, usando texto, números e um valor verdadeiro/falso (booleano) para determinar se há frete grátis.

Código Guiado:

```
funcao gerarDescricaoProduto(nomeProduto, preco, temEstoque) {  
    descricaoBase = "Produto: " + nomeProduto + " | Preço: R$ " + preco  
    // Usamos o parâmetro booleano para uma decisão  
    se (temEstoque == verdadeiro) {  
        retornar descricaoBase + " | Disponível para compra."  
    } senao {  
        retornar descricaoBase + " | Produto esgotado."  
    }  
}
```



```
// Testando os dois casos
```

```
descricao1 = gerarDescricaoProduto("Notebook Gamer", 5000.00,  
verdadeiro)
```

```
escreva(descricao1)
```

```
descricao2 = gerarDescricaoProduto("Mouse sem Fio", 80.00, falso)
```

```
escreva(descricao2)
```

Seu Desafio: Crie uma função `podeEntrarNaFesta(nome, idade)` que retorne verdadeiro se a idade for 18 ou mais, e falso caso contrário. Teste com diferentes idades e exiba o resultado.

Atividade 6:

O Mundo Interno da Função (Escopo de Variáveis)

Objetivo: Entender que variáveis criadas dentro de uma função são locais, temporárias e não podem ser acessadas de fora (escopo local).

Guia Detalhado: Uma função cria uma "bolha" de proteção. As variáveis que você cria dentro dela (`resultadoDaSoma`, `descricaoBase`, etc.) só existem e são visíveis lá dentro. Quando a função termina, elas são destruídas. Isso evita conflitos e organiza o código.

Código Guiado para Testar o Escopo:

```
funcao calcularPrecoFinal(precoBase) {  
    // "taxa" e "valorTaxa" são variáveis de ESCOPO LOCAL.  
  
    taxa = 0.10 // 10% de imposto  
  
    valorTaxa = precoBase * taxa  
  
    precoTotal = precoBase + valorTaxa
```

```
    retornar precoTotal
}

precoCamiseta = 50.00
precoComImposto = calcularPrecoFinal(precoCamiseta)

escreva("O preço final da camiseta é: " + precoComImposto)

// A linha abaixo vai causar um ERRO! Tente executá-la.
// A variável "valorTaxa" não existe aqui fora, ela morreu quando a função
terminou.

escreva("O valor da taxa foi: " + valorTaxa)
```

Seu Desafio: Analise o código e explique com suas palavras por que a última linha causa um erro. O que é o "escopo local"?

Atividade 7:

Deixando seu Código Legível (Documentando Funções)

Objetivo: Aprender a usar comentários para documentar o que uma função faz, seus parâmetros e seu retorno. Isso é uma prática profissional essencial.

Guia Detalhado: Você (ou um colega) pode precisar entender o que uma função faz meses depois de tê-la escrito. Documentar é a chave. Uma boa documentação explica O QUÊ, e não COMO.

Código Guiado:

/*

*** Função: calcularAreaRetangulo**

*** -----**

*** Calcula a área de um retângulo com base em sua largura e altura.**

*** * largura: (número) A largura do retângulo.**

*** altura: (número) A altura do retângulo.**

*** * retorna: (número) A área calculada do retângulo.**

***/**

funcao calcularAreaRetangulo(largura, altura) {

retornar largura * altura

}

Seu Desafio:

Volte nas atividades 3, 4 e 5 e adicione um bloco de comentários de documentação para as funções `saudacaoPersonalizada`, `somar` e `gerarDescricaoProduto`.

Atividade 8:

Funções como Blocos de LEGO (Composição)

Objetivo: Entender o poder de chamar uma função de dentro de outra para criar programas mais complexos e organizados.

Guia Detalhado: Este é o conceito de **composição**. O resultado (retorno) de uma função "bloco" serve como ingrediente (argumento) para outra.

Seu Desafio: Vamos criar um programa que calcula o preço com desconto de um produto e depois adiciona o frete.

Código Guiado:

// Bloco 1: Calcula o valor do desconto

```
funcao calcularDesconto(preco, porcentagem) {  
    retornar preco * (porcentagem / 100)  
}
```

// Bloco 2: Calcula o preço final, usando o Bloco 1!

```
funcao calcularPrecoComDescontoEFrete(precoOriginal,  
porcentagemDesconto, valorFrete) {
```

// 1. Chamamos a primeira função para nos ajudar.

```
    valorDoDesconto = calcularDesconto(precoOriginal,  
porcentagemDesconto)
```

// 2. Usamos o resultado para o cálculo principal.

```
    precoComDesconto = precoOriginal - valorDoDesconto
```

// 3. Finalizamos e retornamos.

```
    precoFinal = precoComDesconto + valorFrete
```

```
    retornar precoFinal  
}
```

```
// Programa Principal
```

```
precoProduto = 200.00
```

```
precoAPagar = calcularPrecoComDescontoEFrete(precoProduto, 10,  
15.00) // 10% de desconto, R$15 de frete
```

```
escreva ("O valor final a pagar é: R$ " + precoAPagar)
```

Para Refletir: Se a forma de calcular o desconto mudasse, em quantas funções você precisaria mexer? Qual a vantagem disso?

Atividade 9:

Tornando Parâmetros Opcionais (Valores Padrão)

Objetivo: Aprender a definir um valor padrão para um parâmetro, tornando sua passagem opcional na chamada da função.

Guia Detalhado: Às vezes, um parâmetro tem um valor que é usado na maioria das vezes. Podemos defini-lo como "padrão". Se a pessoa não informar esse valor na chamada, o padrão é usado.

Código Guiado:

```
// "nivelPermissao" tem um valor padrão de "usuario".
```

```
funcao criarUsuario(nome, email, nivelPermissao = "usuario") {
```

```
    escreva("Usuário criado: " + nome + " | Email: " + email + " | Nível: " +  
    nivelPermissao)
```

```
}
```

// Chamada 1: Não passamos o nível. O padrão "usuario" será usado.

```
criarUsuario("Carlos", "carlos@email.com")
```

// Chamada 2: Passamos um nível. O padrão será ignorado e o nosso valor será usado.

```
criarUsuario("Beatriz", "bia@email.com", "administrador")
```

Seu Desafio: Crie uma função `enviarEmail(destinatario, assunto, corpo, prioridade = "normal")`. Chame-a uma vez sem informar a prioridade e outra vez informando a prioridade como "urgente".

Atividade 10:

Projeto Final - A Calculadora de IMC

Objetivo: Aplicar todos os conceitos aprendidos para construir um mini-programa modular e funcional do zero.

Guia Detalhado: O Índice de Massa Corporal (IMC) é calculado pela fórmula: $\text{peso} / (\text{altura} * \text{altura})$. Vamos criar um programa que solicita os dados do usuário, calcula, interpreta e exibe o resultado.

Seu Desafio: Crie as três funções descritas abaixo e faça-as trabalhar em conjunto.

Esqueleto do Projeto:

// FUNÇÃO 1: Focada apenas no cálculo.

// Deve receber peso (ex: 70) e altura (ex: 1.75) e retornar o valor numérico do IMC.

funcao calcularIMC(peso, altura) {

// TODO: seu código aqui

// Dica: A fórmula é peso dividido por altura ao quadrado.

}

// FUNÇÃO 2: Focada apenas na interpretação.

// Deve receber um número (o IMC) e retornar uma string com a classificação.

// IMC < 18.5 -> "Abaixo do peso"

// IMC >= 18.5 e < 25 -> "Peso normal"

// IMC >= 25 -> "Sobrepeso"

funcao interpretarIMC(valorIMC) {

// TODO: seu código aqui

// Dica: Use uma estrutura se / senao se / senao.

}

// FUNÇÃO 3: A "Coordenadora" do programa.

// Não precisa de parâmetros. Ela vai pedir os dados ao usuário.

funcao iniciarCalculadora() {

escreva("--- Calculadora de IMC ---")

// TODO: Peça para o usuário digitar o peso e guarde em uma variável.

// TODO: Peça para o usuário digitar a altura e guarde em outra variável.

// TODO: Chame a função calcularIMC com os dados do usuário e guarde o resultado.

// TODO: Chame a função interpretarIMC com o resultado do cálculo e guarde a interpretação.

// TODO: Exiba um relatório completo para o usuário.

// Ex: "Seu IMC é [valor] e sua classificação é [interpretacao]."

}

// --- PROGRAMA PRINCIPAL ---

// A única coisa que fazemos aqui fora é dar o "start".

iniciarCalculadora()